

Splash Electronics Admin UI Audit

Generated on: 2026-04-28 Workspace reviewed: d:\Splash-Electronics

Scope

This report documents the current admin-facing UI in the Splash Electronics project, the feature set available on each screen, and the backend or state-management logic that powers those features.

The review was based on the current code in:

- frontend/src/pages/admin/pages/*
- frontend/src/pages/admin/AdminRoutes.jsx
- frontend/src/utils/api.js
- backend/routes/*
- backend/controllers/*
- backend/services/*
- backend/models/*

Admin Foundation

Access control

- The admin area is mounted at /admin/* from frontend/src/App.jsx.
- frontend/src/pages/admin/AdminRoutes.jsx blocks entry unless the user is authenticated and passes isAdminUser(user).
- While authLoading is true, the route shows a full-screen loading state instead of rendering admin content too early.

Shared API behavior

- frontend/src/utils/api.js injects the bearer token from localStorage into every request.
- The same interceptor also adds X-Visitor-Id so analytics and checkout flows can tie browsing and ordering activity together.

Admin shell and navigation

- frontend/src/pages/admin/pages/AdminLayout.jsx provides the permanent admin shell.
- The left sidebar supports collapse/expand and routes to Overview, Products, Categories, Orders, Inventory, Users, Offers, Coupons, Sales Analytics, Forecasting, Returns, Commerce, Settings, and Profile.
- The header includes:
 - a visual search box that is currently not wired to any data source
 - order notifications

- an admin profile menu

Order notifications

- AdminLayout polls GET /api/admin/orders/notifications every 20 seconds for admins.
- A localStorage key named admin_order_notif_last_seen_at is used as the "since" value.
- Opening the order screen or marking notifications as read resets the counter locally.
- The backend returns recent orders created after the stored timestamp, limited by query param.

Overview

UI features

- Executive summary hero and manual refresh button.
- KPI cards for total revenue, total orders, total customers, pending orders, gross sales, cash collected, and refunds issued.
- Best sellers card with product image snapshots, sold quantity, and revenue.
- Order status composition donut.
- Gross sales by division donut.

Logic behind it

- The page calls GET /api/admin/overview.
- backend/controllers/adminController.js computes:
- total order count from Order
- total customer count using admin/customer filters from utils/adminAccess
- recognized sales and cash collected from paid or otherwise collected payment states
- refunds issued from ReturnRefund documents with status refunded
- net revenue as cash collected minus refunds
- order status counts grouped by status
- best sellers by unwinding order items from collected orders
- sales by division from shippingAddress.division
- last 7 days sales summary for recent trend data

Products

UI features

- Catalog dashboard with page-level stats for total products, published products on the current page, featured products on the current page, and low-stock products on the current page.
- Filter shelf with:
- keyword search

- async brand filter
- async category filter
- publication status filter
- featured-only toggle
- published-only toggle
- Catalog table showing image, name, slug, brand, category, price, available inventory, publication status, and actions.
- Row actions:
 - edit
 - duplicate
 - soft delete from storefront
- Full product editor for create and edit flows.
- Featured toggle and publication status selector.
- Base product fields:
 - name
 - slug
 - brand
 - category
 - base price
 - warranty
 - description
 - highlights
 - specs
- Category-driven preset helpers:
 - "Use Category Preset" to inject template highlights and specs
 - "Save As Category Template" to push current highlights/specs back to the selected category
- Variant management:
 - add variant
 - remove variant
 - set default variant
 - edit SKU, price, stock, low-stock threshold
 - edit dynamic variant attributes derived from category metadata
- Existing variant image gallery in edit mode with per-image delete.
- New per-variant image upload staging that happens on final save.
- Footer actions for save, save as draft, and save and publish.
- Quick-create modal for brands and categories without leaving the editor.
- Bulk import modal with:
 - downloadable CSV template
 - CSV file chooser

- grouped product preview
- row validation
- optional auto-create for missing brands/categories
- import feedback including imported image counts

Logic behind it

- The list screen loads GET /api/products/admin with filters for keyword, brand, category, status, featured, and publishedOnly.
- backend/controllers/productController.js enriches admin list results with:
- inventorySummary from services/stockReservationService
- offerPricing from services/offerPricingService
- Product status is driven by publicationStatus with fallback to legacy isActive behavior.
- Soft delete sets isDeleted=true, isActive=false, publicationStatus=archived, and deletedAt, instead of removing the document.
- Duplicating a product:
 - creates a draft copy
 - generates a unique slug
 - creates fresh SKUs
 - resets stock to zero
 - clears review and view metrics
 - clears images
- The editor preserves create drafts in localStorage under admin_product_create_draft_v1 and warns on browser unload if there are unsaved changes.
- While creating a product, the slug auto-follows the name unless manually changed.
- Category selection controls variant attribute fields by reading category.attributes or category preset logic in the frontend.
- Specs text can auto-fill empty attributes on the default variant so structured variant data stays aligned with product-level specs.
- Save validation enforces:
 - required name, slug, brand, category, and description
 - unique SKUs within the product
 - SKU presence for any variant with queued image uploads
 - exactly one default variant
- On save, the frontend first creates or updates the product payload, then re-fetches the saved product and uploads queued variant images by matching each variant through SKU.
- The backend also validates SKU uniqueness across the entire catalog before insert or update.
- Base price is normalized on the backend from the lowest valid variant price whenever variants exist.

- Bulk import logic in AdminProducts:
- parses CSV locally
- groups repeated slug rows into one product with multiple variants
- validates missing description, slug, brand, category, SKU, price, stock, duplicate slugs, duplicate SKUs, and bad image URLs
- can auto-create missing brand/category dependencies
- submits only valid grouped payloads to POST /api/products/bulk-import
- The bulk import backend:
- creates products one by one
- tracks created vs failed rows
- can import remote image URLs into Cloudinary and attach them to variants after creation

Categories

UI features

- Searchable category list.
- Create and edit form for:
- name
- slug
- variant attributes
- highlights template
- specs template
- Category table with edit and delete actions.

Logic behind it

- The page reads GET /api/categories.
- Category create and update send normalized payloads to POST /api/categories and PUT /api/categories/:id.
- backend/controllers/categoryController.js:
- normalizes attribute keys into underscore-style tokens
- de-duplicates string lists
- normalizes specs template keys and values
- stores templates that feed the product editor for new product drafting
- Admin lookups for filters and quick selectors come from GET /api/categories/admin/lookups.

Orders

UI features

- Active fulfillment queue with URL-backed filters for:

- from date
- to date
- status
- payment method
- keyword
- Summary cards for active, confirmed, processing, and shipped counts.
- Paginated order table with order number, customer, payment method/status, region, total, and status pill.
- "View / Update" modal with:
 - line items and pricing breakdown
 - payment summary
 - fulfillment mode selector
 - courier picker
 - courier status selector
 - tracking ID
 - tracking URL
 - booking reference
 - pickup date
 - courier status note
 - internal notes
 - sequential workflow buttons
 - shipment save button
 - shipment booking button
 - delete button for cancelled/returned orders only

Logic behind it

- The main grid calls GET /api/admin/orders with scope=active and the current filters.
- The selected modal loads GET /api/orders/:orderNo for full order detail.
- backend/controllers/orderController.js enforces a strict admin workflow:
 - pending -> confirmed or cancelled
 - confirmed -> processing or cancelled
 - processing -> shipped or cancelled
 - shipped -> delivered
 - delivered -> returned
- Orders cannot skip steps.
- Prepaid orders cannot move into revenue-recognized fulfillment states unless payment.status is already paid.
- Marking an order shipped requires courier and tracking ID.
- Shipment metadata is saved separately through PATCH /api/admin/orders/:orderNo/shipment.

- Workflow status changes are saved through PUT /api/admin/orders/:orderNo/status.
- Dispatch booking is available only for processing orders and calls POST /api/admin/orders/:orderNo/dispatch.
- Dispatch uses the configured courier provider abstraction, writes shipment details, sets shipped timestamps, and creates courier events.
- Stock behavior is tightly coupled to order status:
 - shipping or delivery deducts stock if it has not already been deducted
 - cancellation releases reservations and restocks if stock had already been deducted
 - returned status can auto-create a return/refund request if one does not exist
- Shipment events are written into order.shipment.events so customer and admin tracking UIs can render a timeline.
- Deletion is hard delete but only for cancelled or returned orders, and it also deletes linked ReturnRefund and InventoryLedger rows.
- The order notifications dropdown in the admin shell uses adminGetOrderNotifications and only reports newly created orders after the stored last-seen timestamp.

Inventory

UI features

- Inventory Intelligence dashboard with manual refresh and a global low-stock threshold input.
- KPI cards for total SKUs, units available, on-hand value, and low-stock variant count.
- Donut charts for:
 - stock health composition
 - movement type mix
 - low-stock by category
- Low Availability Queue table showing product, SKU, brand/category, available, threshold, reserved, on hand, and price.
- Right-side manual adjustment form with quick-action presets:
 - Add Stock
 - Remove Stock
 - Correction
 - Return Received
- Recent movement trend chart.
- Recent movement summary list.
- Full movement history table with filters for search text and movement type.
- Full stock table with search and category filter.

Logic behind it

- The page calls GET /api/admin/inventory/overview?threshold=...
- backend/controllers/inventoryController.js:

- loads active, non-deleted products
- walks every variant
- calculates reserved quantity using `services/stockReservationService.getReservedQtyMap`
- calculates sellable available stock as physical on-hand minus reserved
- flags low-stock rows using `variant.lowStockThreshold` with fallback to the requested threshold
- computes total stock value from on-hand multiplied by variant or base price
- loads recent InventoryLedger records with populated product and actor data
- Manual adjustments post to `POST /api/admin/inventory/adjust`.
- The backend rejects:
 - missing product or variant
 - zero delta
 - negative resulting stock
 - any adjustment that would push on-hand below currently reserved stock
- Successful adjustments also create InventoryLedger entries with old/new on-hand, reserved, available, actor, reason, and note.

Users

UI features

- Metrics for total customers, total users, admin count, and newly joined users in the last 30 days.
- Two-tab layout:
 - Users
 - Make an Admin
- Users tab supports:
 - role filter
 - search by name/email/phone
 - pagination
- The Make an Admin tab supports live customer search and one-click promotion.

Logic behind it

- The page calls `GET /api/admin/customers` for both the user list and promotion search.
- `backend/controllers/customerController.js` returns:
 - paginated user rows
 - filtered totals
 - global metrics across all users
- Role updates call `PATCH /api/admin/customers/:userId/role`.
- The backend prevents:

- invalid role values
- demoting yourself through this screen
- removing the last admin account
- promoting a user who is already admin or demoting a user who is already customer becomes a no-op success response

Offers

UI features

- Search and status filter for active, upcoming, expired, or disabled offers.
- Metrics for total, enabled, upcoming, and expired offers.
- Create/edit form for:
 - name
 - label
 - description
 - discount type
 - value
 - priority
 - scope
 - audience
 - valid from/to
 - active toggle
- Async scope pickers for products, categories, and specific users.
- Offer list with edit and deactivate actions.

Logic behind it

- The page calls GET /api/admin/offers.
- Scope pickers use:
 - GET /api/admin/offers/lookups/products
 - GET /api/admin/offers/lookups/categories
 - GET /api/admin/offers/lookups/users
- The frontend also has fallback lookups if those endpoints are unavailable.
- backend/controllers/offerController.js supports list, create, update, and soft delete.
- An offer can target:
 - all products
 - selected products
 - selected categories
 - and optionally all users or selected users
- Status is derived from isActive plus the validFrom/validTo window.
- Deactivate does not remove the record; it sets isActive=false.

- services/offerPricingService.js applies active offers to product and variant pricing at read time.
- If multiple offers match the same product, the service chooses the best effective price, then breaks ties by higher priority and newer creation time.

Coupons

UI features

- Search and status filter.
- Metrics for total, active-flag, upcoming, and expired coupons.
- Draft restore banner when a coupon form draft is recovered from localStorage.
- Create/edit form for:
 - code
 - auto-generate code
 - coupon type
 - discount value
 - max discount
 - min cart total
 - usage limit
 - per-customer usage limit
 - valid from/to
 - customer eligibility
 - discount applies to
 - active toggle
 - internal description
- Async selectors for specific products, categories, and users.
- Coupon list with status badge, validity window, usage stats, edit, and deactivate.

Logic behind it

- The page saves local draft state in localStorage under admin_coupon_form_draft_v1.
- Coupon list calls GET /api/admin/coupons.
- Code generation uses GET /api/admin/coupons/generate-code and derives a prefix hint from the current form.
- Coupon CRUD uses POST, PUT, and DELETE under /api/admin/coupons.
- backend/controllers/couponController.js enforces unique codes, normalizes FLAT into FIXED, and soft-deactivates on delete.
- Coupon applicability data includes:
 - product scope
 - category scope
 - specific users
 - new-customer only

- returning-customer only
- usage caps
- per-customer caps
- In checkout, backend/services/orderService.js validates coupon rules against:
- active flag
- valid date window
- total usage cap
- per-customer usage cap
- specific users
- new vs returning customer eligibility
- eligible products/categories in the cart
- min cart total or eligible-items subtotal
- discountAppliesTo logic for eligible items only vs entire cart

Sales Analytics

UI features

- Date range picker with default last 30 days.
- KPI cards for revenue, orders, average order value, unique customers, unique viewers, conversion rate, abandoned carts, and peak order time.
- Daily revenue trend chart.
- Payment-method donut chart that can jump into the report table with that payment method filter applied.
- Finance cards for gross sales, cash collected, and refunds issued.
- Charts for:
 - gross sales by division
 - product orders by division
 - top products by revenue
 - most viewed products
 - cash collected by payment method
 - peak order hours
- Sales Report section with:
 - finance KPIs
 - filterable read-only order table
- CSV export across all pages

Logic behind it

- The analytics cards call GET /api/admin/analytics/overview.
- backend/controllers/analyticsController.js calculates:
 - order totals in the selected date range
 - unique customers

- gross sales and collected cash from collected payment states
- refunds issued from refunded return rows
- net revenue
- daily paid revenue
- sales and product-order mix by division
- payment method mix
- peak order hours
- product views and unique viewers from ProductView
- ordering visitors from order analytics.visitorKey
- approximate conversion rate
- abandoned carts by looking for carts that were updated but never followed by an order
- The Sales Report table uses GET /api/admin/orders with scope=finance.
- In finance scope, backend order reporting merges:
 - order payment collection timing
 - refund events tied to the same order
 - calculated net revenue per order
- CSV export requests all pages in batches of 500 and builds a client-side CSV file.

Forecasting

UI features

- History selector for 30, 60, 90, or 180 days.
- Forecast horizon selector for 7, 14, 30, or 60 days.
- KPI cards for forecast revenue, forecast units, page views, unique viewers, average conversion, trending SKUs, projected risk SKUs, and confidence.
- Lead product spotlight.
- Methodology explainer cards.
- Signal overview panel.
- Demand momentum line graphic.
- Category revenue share donut.
- Ranked top-forecasted products.
- Category outlook cards.
- Demand Intelligence table.
- Stockout Risk and Reorder Plan table.

Logic behind it

- The page calls GET /api/admin/analytics/forecasting?daysBack=...&horizonDays=...
- backend/controllers/analyticsController.js blends:
 - order quantities and revenue

- recent vs prior demand momentum
- page views and unique viewers
- visitor-to-buyer conversion estimates
- traffic-vs-order weighting
- current stock
- safety stock buffer
- The forecast returns:
 - product-level forecast rows
 - category-level rollups
 - summary KPIs
- model metadata including version and safety stock days
- Risk labels are derived from projected stock remaining after the forecast horizon.
- Suggested reorder quantity equals forecast demand plus safety stock minus current stock.

Returns And Refunds

UI features

- Queue summary cards for total requests, open queue, urgent cases, and refund exposure.
- Search and status filter.
- Left-side request queue cards with SLA-style age labels.
- Right-side detail panel with:
 - customer identity and refund preference
 - returned items
 - evidence photos
 - financial snapshot
 - stage tracker
 - recommended next-step buttons
 - admin notes display

Logic behind it

- The admin page calls GET /api/returns/my.
- That route looks customer-specific by name, but backend/controllers/returnRefundController.js intentionally returns all rows when req.user.isAdmin is true.
- Status updates use PUT /api/returns/admin/:id/status.
- Allowed states are:
 - requested
 - approved
 - rejected

- picked
- received
- refunded
- When a return is marked received, the backend restocks each returned variant and writes InventoryLedger IN entries with reason RETURN.
- When a return is marked refunded, the backend updates the parent order payment status to refunded or partial_refund depending on the amount.

Commerce

UI features

- Checkout and Shipping settings dashboard.
- Summary cards for COD, inside Dhaka, outside Dhaka, and free shipping.
- Order and Checkout section for:
 - Cash on Delivery toggle
 - max COD amount
 - auto-confirm online payments toggle
 - order prefix field
- Shipping Pricing section for:
 - inside Dhaka fee
 - outside Dhaka fee
 - express surcharge inside Dhaka
 - express surcharge outside Dhaka
 - free shipping threshold
- Regional Overrides section for division/district-specific shipping fees.

Logic behind it

- The page reads and writes the same single settings document through GET/PUT /api/admin/settings.
- backend/models/Settings.js stores order and shipping settings in nested objects.
- backend/services/orderService.js reads shipping settings in getShippingConfig and getShippingQuote.
- Checkout shipping calculation uses:
 - Dhaka vs outside-Dhaka baseline fees
 - optional express surcharge
 - optional regional override by division or district
- The UI explains that an empty district means the override applies to the full division.

Important note

- The orderPrefix field is saved in Settings, but order numbers are still generated by backend/utils/orderNo.js with a hard-coded SPL prefix.

- That means the current Admin Commerce UI exposes an order-prefix setting that is not yet wired into order creation.

Settings

UI features

- Brand, storefront, and maintenance settings page.
- Summary cards for store name, support email, primary color, and maintenance status.
- Store Profile form fields for:
 - store name
 - logo URL
 - support email
 - support phone
 - support hours
 - address lines
 - city
 - district
 - country
- UI and Maintenance section for:
 - primary color
 - secondary color
 - homepage banner text
 - announcement bar text
 - maintenance toggle
 - maintenance message

Logic behind it

- The page reads and writes GET/PUT /api/admin/settings.
- backend/controllers/adminSettingsController.js ensures a single Settings document exists and merges partial updates into store, order, shipping, ui, and maintenance objects.
- Public storefront consumption happens through GET /api/settings/public.
- frontend/src/pages/Home.jsx already uses:
 - storeName
 - logoUrl
 - supportEmail
 - supportPhone
 - supportHours
 - address
 - homepageBannerText
 - announcementBarText

- maintenanceEnabled
- maintenanceMessage

Important note

- primaryColor and secondaryColor are stored and exposed through the public settings API, but the current storefront code does not apply them as live theme variables.
- They behave as saved config fields today, not as active visual theming.

Profile

UI features

- Current admin profile card.
- Identity fields for name, email, phone, role, admin access, and last login.
- Raw account ID display.

Logic behind it

- This page is client-side only and reads from UserContext.
- There is no separate admin profile update flow yet.

Notable Cross-Cutting Observations

- The admin shell search input in AdminLayout is visual only and does not trigger any search logic.
- The returns page route naming is slightly misleading: admins use /api/returns/my but receive all returns because the controller checks req.user.isAdmin.
- Product, order, inventory, and analytics modules are well-connected to the same underlying order, stock reservation, and ledger systems, so admin data stays internally consistent.
- Product availability in admin listings is reservation-aware because inventory enrichment subtracts reserved quantities from physical stock.
- Offers and coupons are separate systems:
 - offers are automatic read-time price reductions on products and variants
 - coupons are checkout-time discount rules validated against cart contents and user history

Final Summary

The admin UI is already a broad operations console rather than just a CRUD back office. It covers catalog management, inventory control, fulfillment workflow, returns, promotions, customer access, shipping policy, store messaging, analytics, and forecasting.

The strongest parts of the implementation are:

- the product studio
- the inventory plus reservation-aware stock model
- the sequential order workflow
- the analytics and forecasting coverage

The main implementation gaps observed during this re-check are:

- admin shell search is not wired
- order prefix is configurable in UI but not used by order number generation
- saved storefront colors are not currently applied as runtime theme styling